UNIVERSITY OF DHAKA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT 02

Wireshark Lab: HTTP Protocol Analysis & Packet Inspection

Reference: Kurose & Ross (9th Ed., v9.0)

ASSIGNMENT OVERVIEW & TARGET RESOURCES

Pre-Lab Setup: Clear browser cache prior to each capture

In this laboratory assignment, you will use the Wireshark packet sniffer to investigate the Hypertext Transfer Protocol (HTTP) in operation. You will analyze six distinct scenarios: basic GET/response interactions, conditional caching headers, TCP segmentation of large HTML documents, multi-server object retrieval, HTTP Basic Authentication, and an independent live web server evaluation. Ensure your browser cache is cleared prior to each task and filter Wireshark packet listings using the lowercase display filter `http` (unless inspecting lower-layer TCP segments).

Task	Investigation Topic	Target URL / Trace Resource	Questions
Task 1	Basic HTTP GET / Response	`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html`	Q1 – Q7
Task 2	Conditional GET & Caching	`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html`	Q8 – Q11
Task 3	Retrieving Long Documents	`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html`	Q12 – Q15
Task 4	Embedded Objects	`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html`	Q16 – Q17
Task 5	HTTP Authentication	`http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html`	Q18 – Q19

TASK 1: THE BASIC HTTP GET/RESPONSE INTERACTION

Trace fallback: http-wireshark-trace1-1

EXPERIMENTAL PROCEDURE

1. Launch your web browser and start the Wireshark packet sniffer (do not begin packet capture yet).
2. Enter `http` in the Wireshark display-filter-specification window so only HTTP messages are displayed.
3. Wait slightly more than one minute, begin Wireshark packet capture, and navigate to: `http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html`
4. Once your browser displays the simple one-line HTML file, stop Wireshark packet capture. (Ignore any automatic browser requests for `favicon.ico`; expand the Hypertext Transfer Protocol node while keeping Frame, Ethernet, IP, and TCP collapsed).

1. **HTTP Version:** Is your browser running HTTP version 1.0, 1.1, or 2? What version of HTTP is the server running?
2. **Accepted Languages:** What languages (if any) does your browser indicate that it can accept to the server?
3. **IP Addressing:** What is the IP address of your computer? What is the IP address of the `gaia.cs.umass.edu` server?
4. **Response Status:** What is the status code returned from the server to your browser?
5. **Modification Timestamp:** When was the HTML file that you are retrieving last modified at the server? *(Note: The*

TASK 2: THE HTTP CONDITIONAL GET/RESPONSE INTERACTION

Trace fallback: http-wireshark-trace2-1

EXPERIMENTAL PROCEDURE

1. Start up your web browser and ensure your browser's cache is completely cleared.
2. Start up the Wireshark packet sniffer and begin capturing network traffic.
3. Enter the following URL into your browser to display a five-line HTML file: `http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html`
4. Quickly enter the exact same URL into your browser again (or click the browser's **Refresh** button).
5. Stop Wireshark packet capture and enter `http` in the display-filter-specification window.

8. **Initial GET Inspection:** Inspect the contents of the first HTTP GET request from your browser to the server. Do you see an `IF-MODIFIED-SINCE` line in the HTTP GET?
9. **Initial Server Response:** Inspect the contents of the server response. Did the server explicitly return the contents of the file? How can you tell?
10. **Second GET Inspection:** Now inspect the contents of the second HTTP GET request from your browser to the server. Do you see an `IF-MODIFIED-SINCE:` line in the HTTP GET? If so, what information follows the `IF-MODIFIED-SINCE:` header?
11. **Cache Validation Response:** What is the HTTP status code and phrase returned from the server in response to this second HTTP GET? Did the server explicitly return the contents of the file? Explain.

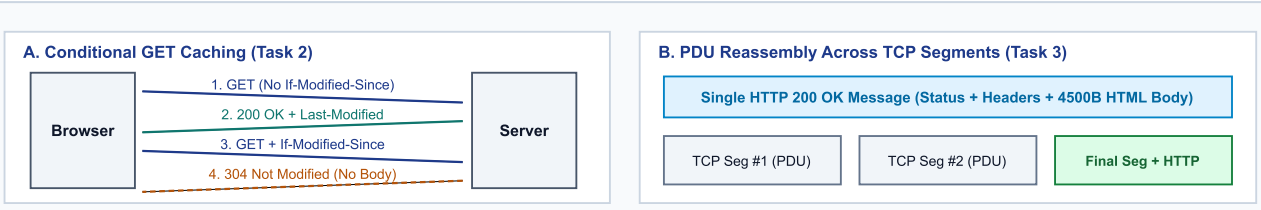


Figure 1: Protocol behavior in Task 2 (Conditional GET cache validation) and Task 3 (TCP segmentation of a 4,500-byte HTTP entity body).

TASK 3: RETRIEVING LONG DOCUMENTS (MULTI-PACKET TCP)

Trace fallback: http-wireshark-trace3-1

EXPERIMENTAL PROCEDURE

1. Clear your browser's cache, start the Wireshark packet sniffer, and retrieve the US Bill of Rights (4,500 bytes): `http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html`
2. Stop Wireshark packet capture and enter `http` in the display-filter window to locate the HTTP GET and response.
3. **Clear the Wireshark display filter** so that the individual data-containing TCP segments (`TCP segment of a reassembled PDU`) carrying the fragmented response are visible in the packet listing.

12. **GET Packet Identification:** How many HTTP GET request messages did your browser send? Which packet number in the trace contains the GET message for the Bill of Rights?
13. **Response Packet Identification:** Which packet number in the trace contains the status code and phrase associated with the response to the HTTP GET request?
14. **Response Status Line:** What is the status code and phrase in the response?
15. **TCP Segmentation Count:** How many data-containing TCP segments were needed to carry the single HTTP response and the text of the Bill of Rights?

TASK 4: HTML DOCUMENTS WITH EMBEDDED OBJECTS

Trace fallback: http-wireshark-trace4-1

> **EXPERIMENTAL PROCEDURE**
>
> 1. Clear your browser's cache, start Wireshark packet capture, and enter the following URL into your browser: `http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html`
> 2. Your browser displays a short HTML file referencing two external images (the publisher logo on `gaia.cs.umass.edu` and the 8th edition book cover hosted on a server in France).
> 3. Stop Wireshark capture and apply the `http` display filter to inspect all outbound HTTP GET requests.

16. **Multi-Object GET Requests:** How many HTTP GET request messages did your browser send? To which Internet (IP) addresses were these GET requests sent?
17. **Serial vs. Parallel Download:** Can you tell whether your browser downloaded the two images serially, or whether they were downloaded from the two web sites in parallel? Explain using packet timestamps and request-response ordering.

TASK 5: HTTP AUTHENTICATION & BASE64 ENCODING

Trace fallback: http-wireshark-trace5-1

> **EXPERIMENTAL PROCEDURE**
>
> 1. Clear your browser's cache, close and restart your browser, and start Wireshark packet capture.
> 2. Navigate to the password-protected URL: `http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html`
> 3. When prompted by the authentication pop-up box, enter username `wireshark-students` and password `network` (without quotation marks), then stop capture and filter with `http`.

18. **Initial Challenge Response:** What is the server's response (status code and phrase) in response to the initial HTTP GET message from your browser?
19. **Credential Transmission & Base64 Decoding:** When your browser sends the HTTP GET message for the second time, what new field is included in the HTTP GET message? Decode the Base64 string (`d2lyZXNoYXJrLXN0dWRlbnRzOm5ldHdvcms=`) following `Authorization: Basic` into ASCII and explain why HTTP Basic Authentication without TLS is insecure against packet sniffers. You can use any online Base64 decoder to decode the encrypted string. As for example, Base64 Decode and Encode - Online.